



Module 1-2

Digital IC Design Flow Overview (Part 1)

cādence®

This module spans 3 hours of lecture time.

Module Objectives

In this module, you will:

- Draw a flow diagram of the entire design flow.
- Identify the distinction between Digital IC design, verification, and implementation.



In this module, you will:

- Draw a flow diagram of the entire design flow.
- Identify the distinction between Digital IC design, verification, and implementation.
- Recognize the different stages of front-end design and verification.
- Recognize the different stages of design implementation.
- Identify the challenges of scaling, costs, and physical attributes, as well as low power and area constraints before tapeout.
- Identify the different processes in the semiconductor industry used to handle the above realistic challenges.

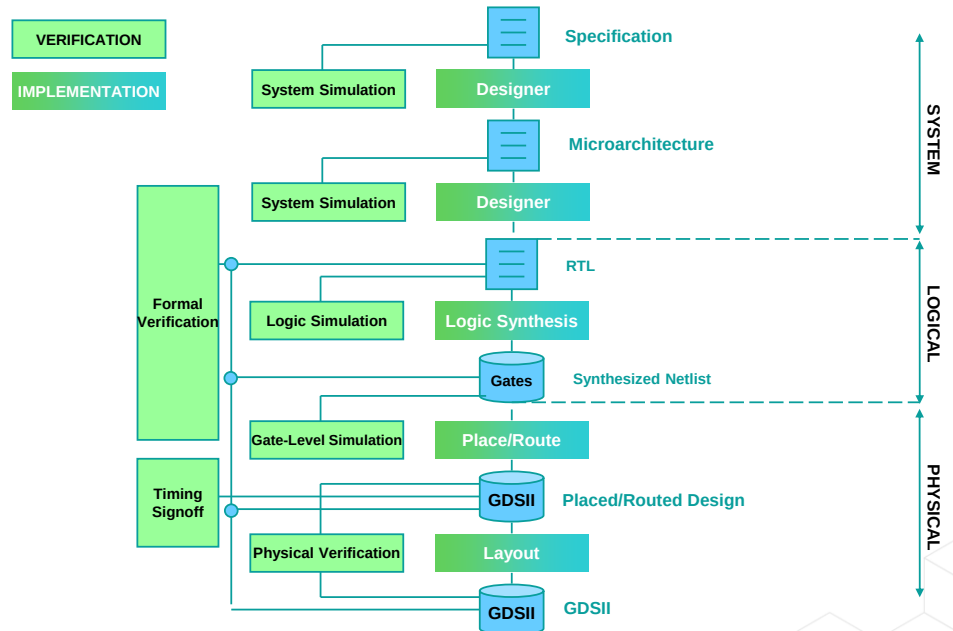
Complete Basic IC Design Flow

A design flow can be divided into three phases:

- System
- Logical
- Physical

In each phase, two main processes need to be performed:

- Implementation
- Verification



3 © Cadence Design Systems, Inc. All rights reserved.

Let us discuss briefly the three phases of design, the system, the logical, and the physical.

In the system phase, you define the requirements and develop the specifications for your design. You then develop the micro-architecture and run simulation to test your system for various parameters to deduce the expected power, area, and timing. When that phase is satisfactory, you move on to the logical phase.

In the logical phase, you develop the (Register Transfer Logic) RTL. Intellectual property (IP) reuse is common in this step. The RTL must then be simulated to verify the desired functionality.

The RTL must then be synthesized to gates from the technology of the foundry you desire to target.

The gates are transferred in the form of a netlist into the place and route tool, where you run floorplanning, placement and routing.

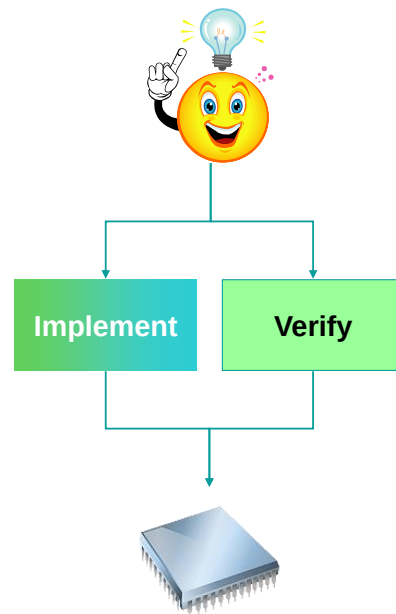
After one more round of verification at the physical level, and after meeting all your signoff requirements, you then proceed to generate a, G D S 2, so as to create a mask for the foundry.

Also remember that in each of these phases of the, system, logical and physical, we just discussed, there are 2 processes which run all the time, one is Verification and the other is Implementation.

Two Main Processes of the IC Design Flow

At the highest level, what tasks need to be performed when creating a chip from an idea?

- Implementation
 - Transform the design from idea or specification to various representations of logical and physical hardware.
- Verification
 - Ensure the functionality, timing, and integrity of the changing design through the process.



4

© Cadence Design Systems, Inc. All rights reserved.

cadence

The current day digital systems such as 5G networks, related smartphones, compute farms, cloud computing, edge computing, artificial intelligence (A I) machines, autonomous cars etc. all started with a specification.

Each system must be thoroughly verified to provide the correct functionality, but also to achieve the proper balance of power and performance.

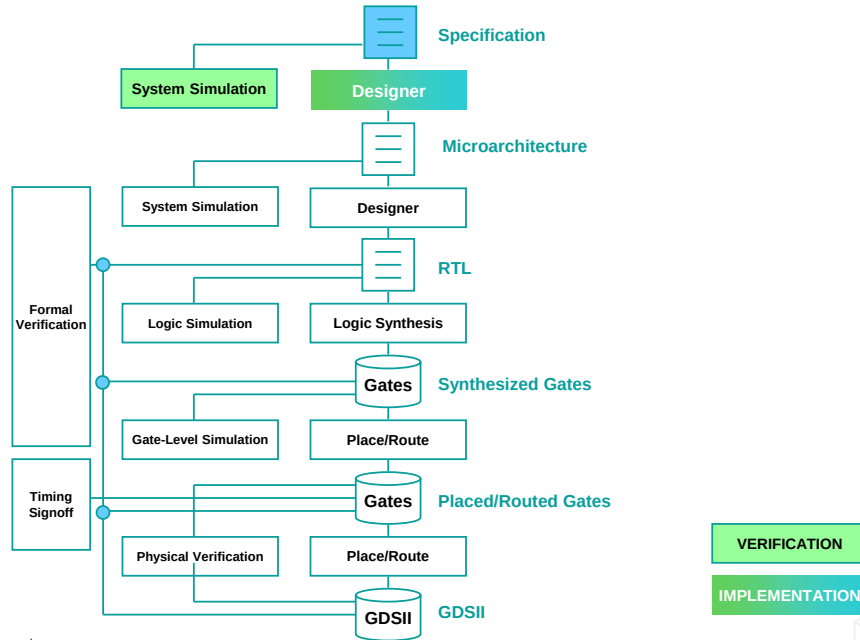
Many trade-offs are made along the way to achieve an optimal result.

So, At the highest level, what tasks need to be performed when creating a chip from an idea?

Implementation, that is, to transform the design from idea or specification to various representations of logical and physical hardware.

Verification, which is, to ensure the functionality, timing, and integrity of the changing design through the process.

It Starts with a Specification



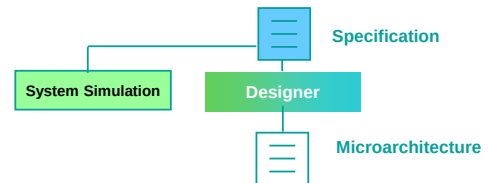
5 © Cadence Design Systems, Inc. All rights reserved.

The idea begins in the form of a specification. It could start from a simple spreadsheet or a doc or a pdf and evolve into a complex analysis using a chip planning solution. Usually in an organized environment in the semiconductor industry, the Design Architect is responsible for writing the design specification.

What Is a Specification?

A specification is an explicit set of requirements to be satisfied by a material, product, or service.

- Ideas begin with a specification, which can be a textual, graphical, or even a software representation.
- For chip design, the specification is the reference model the team uses to:
 - Design the overall chip.
 - Specify the intellectual property used.
 - Specify the “new logic” to be created.
 - Specify the block-level and chip-level interfaces.
 - Partition the chip into functional blocks.
 - Communicate interfaces and requirements with other teams.
 - Measure actual performance versus specified targets.
- **Example:** The specification for the latest chip called for a 250-MHz core clock with a 6G SERDES interface, able to process 1M streams of data per second at less than 10W total power.



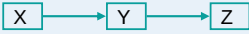
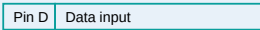
Let us understand what a Design Specification is. It is a set of explicit requirements to be satisfied by a material, a product or a service.

It can conceptualize as an idea which can be textual, graphical or even a software representation.

For chip design, the specification is the reference model the team uses to:

- Design the overall chip.
- Specify the intellectual property used.
- Specify the “new logic” to be created.
- Specify the block-level and chip-level interfaces.
- Partition the chip into functional blocks.
- Communicate interfaces and requirements with other teams.
- Measure actual performance versus specified targets.

Example: Specification

Title	Specification for XYZ Design
List of Reviewers	HW, SW, Test, Manufacturing, Customer, etc.
Modification History	1.0 – 12/2007 – Initial Revision, 2.0 – 1/2008 – Changes to ...
Table of Contents	Section 1 Overview – Section 2 Block 1 ...
Glossary	XYZ = Codename for project, etc.
Overview	The XYZ chip is a new product targeted for consumers...
Performance Targets	125 MHz, 1W Total power, 1M streams/sec
Block Diagrams	 <pre> graph LR X --> Y --> Z </pre>
Graphs	
Tables	
Detailed Description	Block A is input block, it receives signals from the I/O...

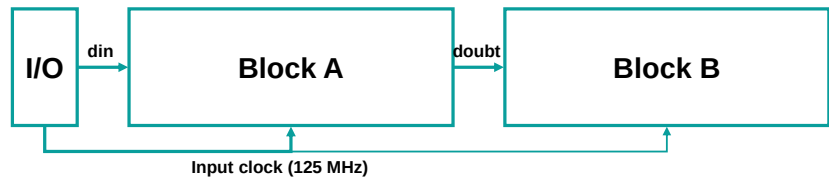
This slide shows a table containing the items that should be part of a specification document.

First is the title of the specification, in this case, the X Y Z Design, then the list of reviewers , or the stakeholders we can say, The Modification history if at all it has been modified in the past, the table of contents of the document, the Glossary, the short intro or overview of the design, The Performance target parameters like the frequency, power and speed. The design block diagrams, then graphs if any to depict the functionality, tables of values if any, and then the detailed description of the entire design.

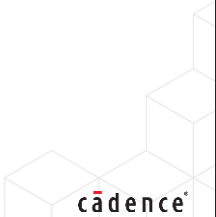
Specification Snippet Example

Block A Interface

Block A is the input block, which receives serial data from the I/O and transfers to Block B.



Port Name	Direction	Source/Destination	Size	Description
clk	input	I/O	1 bit	Clock at 800 MHz
din	input	I/O	32 bits	Input data
dout	output	A and B	32 bits	Output data



This slide shows the way a block is described at a higher level in the specification. An example of Block A is described here. A short intro of Block A, followed by the block diagram and the table of important values associated with the Block.

In this example, it says Block A is the input block, which receives serial data from the I/O and transfers it to Block B. The input clock frequency is 125 MHz.

High-Level Decisions

In creating or modifying the specification, high-level decisions *that affect the system and its environment* are made.

- For example, the choice between external SRAM or DRAM:
 - Control for each one is drastically different.
 - I/Os for the chip will be affected.
 - Board itself will be affected, components plus signal routing.
- Another example is the choice to run the design at 250 MHz or 500 MHz:
 - Chip-level clock input has to change.
 - Software might have to change because the performance could be 2X different.



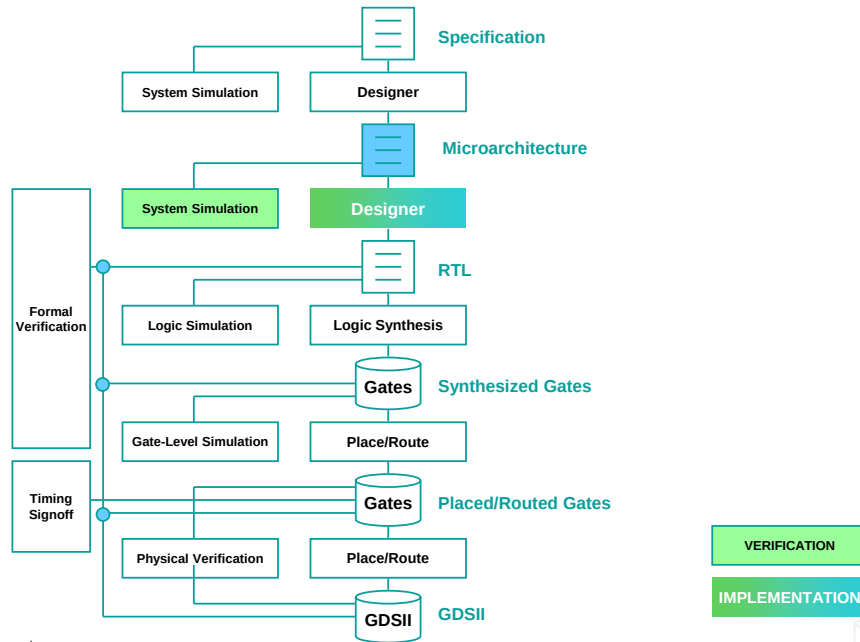
Higher-level decisions about creating a specification are made by the Architect and a team of Stakeholders together. These are the kind of decisions that affect and decide the system and its environment.

For example, the choice between external SRAM and DRAM. Choosing any of these will have a different control for each and will be drastically different from the other.

Further, the I/Os of the chip will be affected, and the board itself will be affected, as well as the components and the signal routing too.

Another example is the choice to run the design either at 250 MHz or 500 MHz. For this, the chip-level clock input has to change. Also, the software might have to change because the performance could be 2X different.

Microarchitecture



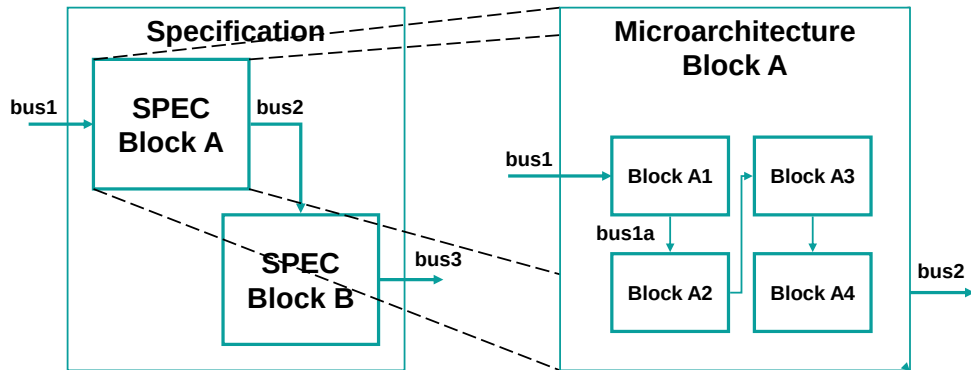
10 © Cadence Design Systems, Inc. All rights reserved.

cadence

The next step in the design flow is converting the high-level design specification to provide more details for the individual design components and blocks and defining the inter-communication amongst them. This is termed as the microarchitecture.

Example: Microarchitecture

The microarchitecture is typically based on a block in the specification.



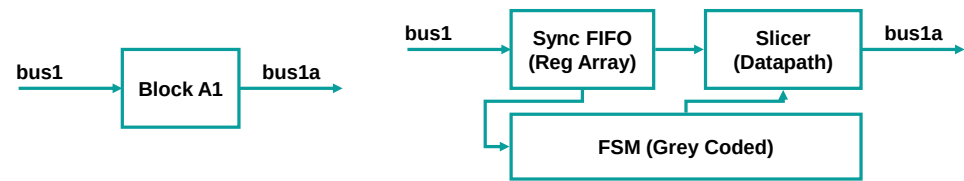
The microarchitecture is typically based on individual blocks in the specification. As seen in the diagram on this slide, the design specification defines Block A and Block B in the spec.

The microarchitecture for Block A, then defines the inner block A1, A2, A3 and A4 and the intercommunication between them. Also, it provides the protocol of Bus1 and Bus2 connecting Block A to other components or parts in the design.

Example: Microarchitecture (continued)

Section 2.1.1.1 Block A1

This is the input block for Block A. Its function is to process and slice the data into 16-bit segments, based on control signals from the FSM, and send it to block B2.



Port Name	Direction	Source/Destination	Size	Description
clk	input	I/O	1 bit	Clock at 125 MHz
bus1	input	I/O and Sync FIFO	32 bits	Input data
bus1a	output	Slicer	16 bits	Output data



This slide shows the microarchitecture description snippet of Block A inside the main specification. It describes its input signals and its functionality. It explains that Block A’s function is to process and slice the data into 16-bit segments, based on control signals from the F S M and send it to Block B2.

This information is depicted as a block diagram, as seen.

There is a table of information provided with values that must be used for running this block, like, the clock frequency of the buses, the signal directions and the size of the signals, etc.

Mid-Level Decisions

In creating or modifying the microarchitecture, mid-level decisions that affect the block itself are made.

- For example, the choice between internal SRAM or register array:
 - Interface to the outside environment is the same.
 - Performance of the block may vary slightly, but functionality is the same.
- The choice to use multiple datapaths versus a single datapath:
 - Area is a tradeoff versus performance.
 - As long as the performance targets are met, how the design is actually implemented is a microarchitectural decision.



As the high-level decisions are made at the Design Spec level, there are mid-level decisions that are made at the Microarchitecture level. These are the decisions for creating or modifying the microarchitecture at the block level.

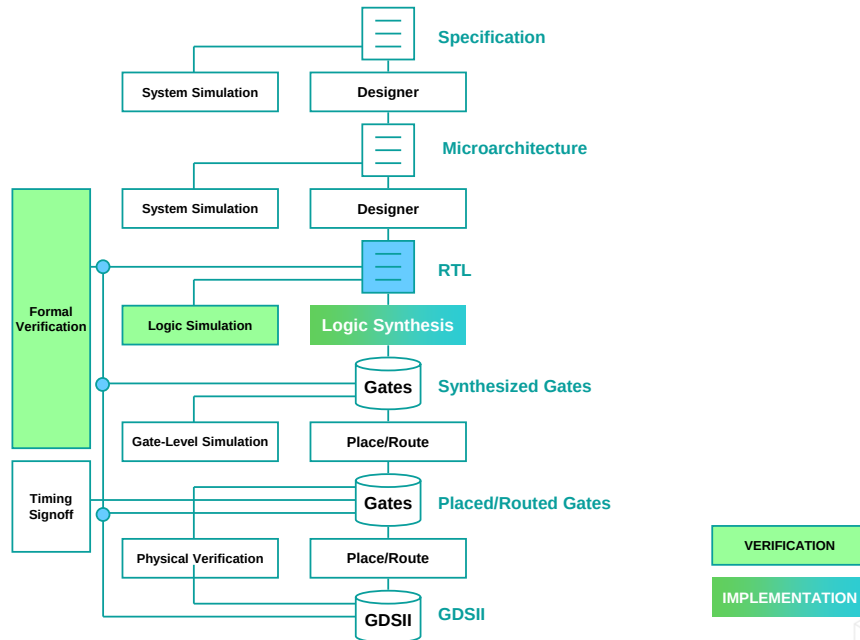
For example, to choose between SRAM and a register array. For each of these choices, the interface to the outside world might be the same, but the performance of the block may vary slightly, even though the functionality remains the same.

Another example of a decision would be to use multiple datapaths versus a single datapath.

In this case, there is a tradeoff between area and performance.

As long as the performance targets are met, how the design is actually implemented is a microarchitectural decision.

Register Transfer Level (RTL) Phase



14 © Cadence Design Systems, Inc. All rights reserved.

cadence

After the main Design Spec is written and also the microarchitecture block-level details are finetuned, then the designer converts his or her design blocks into behavioral modeling or register transfer level modeling through the high-level design language or HDL such as Verilog and SystemVerilog. And high-level verification language or HVL, such as Verilog and SystemVerilog, are used for creating the verification testbench and its related infrastructure.

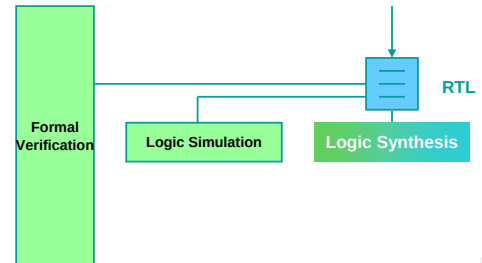
The design code created is “tested” using the testbench written. This phase is called the logic simulation.

What Is RTL?

A way of describing the operation of a digital circuit where the behavior is defined in terms of the flow of signals between registers and the operations performed.

For chip design, the RTL is the reference model the designer uses to:

- Design the block for final implementation.
- Instantiate and connect intellectual property.
- Code the “new logic.”
- Create the block-level interfaces.
- Partition the block into sub-blocks.
- Verify the interfaces to other blocks.
- Run simulations to measure actual performance versus specified targets.
- The translation of a system specification to RTL is a difficult and time-consuming task.



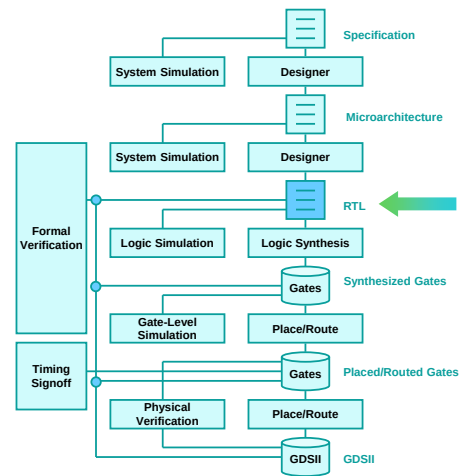
RTL stands for Register Transfer Level. It is a way of describing the operation of a digital circuit where the behavior is defined in terms of the flow of signals between registers and the operations performed.

For chip design, the RTL is the reference model the designer uses to:

- Design the block for final implementation.
- Instantiate and connect intellectual property.
- Code the “new logic.”
- Create the block-level interfaces.
- Partition the block into sub-blocks.
- Verify the interfaces to other blocks.
- Run simulations to measure actual performance versus specified targets.

Importance of RTL in the Flow

- RTL is the representation that bridges the specification and microarchitecture to the final implementation.
- Most RTL is manually created by skilled and experienced design engineers, using the specification and microarchitecture.
- The quality of the RTL directly affects the end product:
 - Time
 - Effort
 - Money



RTL, as described in the previous slides, is the format in which the design blocks are coded. It is the representation that bridges the specification and microarchitecture to the final implementation.

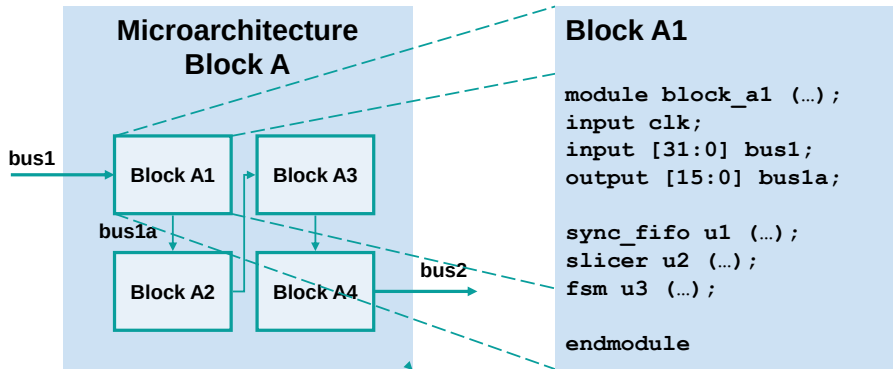
Most RTL is manually created by skilled and experienced design engineers, using the specification and microarchitecture.

The quality of the RTL directly affects the end product in terms of:

- Time, effort and money.
- A good design coding will prove to be a great cause for achieving all these 3 factors.

Example: RTL

The RTL is typically based on a block in the microarchitecture.



17 © Cadence Design Systems, Inc. All rights reserved.

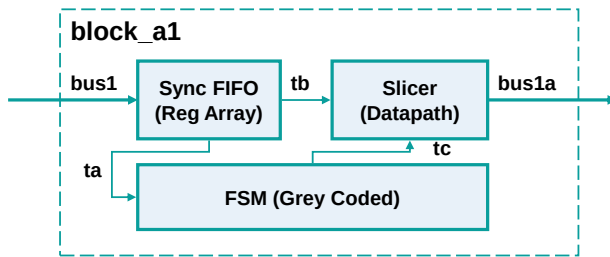


The RTL is typically based on a block in the microarchitecture.

Each block is coded separately and integrated using the designated HDL as decided by the project.

Also, the bus protocols and bus interfaces are also defined for intercommunication between the blocks.

As you can see in this slide, Block A1 is coded as a module, and its structure defined within it.

Example: RTL (continued)

```
// File : block_a1.v
module block_a1 (...);
input clk;
input [31:0] bus1;
output [15:0] bus1a;
sync_fifo u1 (...);
slicer u2 (...);
fsm u3 (...);
endmodule
```

```
// File : sync_fifo.v
module sync_fifo (clk, bus1, ta, tb);
...
endmodule
```

```
// File : slicer.v
module slicer (clk, tb, tc, bus1a);
...
endmodule
```

```
// File : fsm.v
module fsm.v (clk, ta, tc);
...
endmodule
```



Further details for writing a Design Block is described in this slide.

Block A1 consists of 3 blocks, as you can see in the block diagram.

The Sync FIFO Register Array, the Slicer, which is the datapath, and the Finite State machine or FSM, which uses the grey coding style for coding the logic.

These 3 separate sub-blocks are created in 3 separate files, and the logic is defined in modules.

Then, the overall 4th file, with the Block A1 module, is created with the instantiation of the 3 sub-blocks within it.

This is how the block-level microarchitecture details are coded.

Low-Level Decisions

In creating or modifying the RTL, low-level decisions *that affect the implementation of the block itself* are made.

- For example, the choice to use a particular coding style:
 - Designer has previous knowledge of an optimal style for implementation or verification.
 - Designer is more comfortable with a particular style.
- The choice to add pipeline stages versus forcing more logic into a single cycle:
 - Cycle time is traded off for sequential area.
 - As long as the performance targets are met, how the design is actually implemented is a microarchitectural decision.
 - It is possible that the latency of the top-level block is “flexible.”



We already talked about high-level and mid-level decisions. As we go into the details of each block, we have to make more detailed low-level decisions.

These decisions will affect the implementation of the blocks.

Let's look at some examples.

The choice to use a particular coding style would be:

- Designer has previous knowledge of an optimal style for implementation or verification.
- Designer is more comfortable with a particular style.

The choice to add pipeline stages versus forcing more logic into a single cycle is based on factors like:

- Cycle time is traded off for sequential area.
- As long as the performance targets are met, how the design is actually implemented is a microarchitectural decision.
- It is possible that the latency of the top-level block is “flexible.”

Test Your Understanding

Specification/Microarchitecture/RTL

- Who creates the ____?
- What information is required to come up with the details of the ____?
- What other decisions would be considered ____?
- How would we validate these decisions?

	Specification	Microarchitecture	RTL
Who?	CEO, CTO, Marketing, Chip Lead, etc.	Chip Lead, Block-Level Designer	Block-Level Designer
What information?	Customers, Market Data, Competitive Data, etc.	Performance Targets, Block I/O, etc.	Performance Targets, Block I/O, etc.
Other decisions?	Overall architecture, Use of specific IP	Block Partitioning, Memory size and Quantity	Instantiate vs. Infer, Re-use code vs. create
Validate?	System Simulation	System Simulation	RTL Simulation



There are more details to be captured once we start coding the design blocks.

From the Main Design specification to the architecture and then to the RTL level, we need to capture details as shown in the table.

At the main specification level, who is writing the specification? What information goes into this? What decisions need to be made at this level, and what kind of validation is required at this phase in the design cycle?

Then at the microarchitecture level, is the chip lead or the block-level designer who would write the specification? What information is covered in the blocks, like the performance targets, Block I/Os, etc.? What kind of decisions need to be made here, like block partitioning, memory size and quality, etc? And then what kind of validation is needed to test all these at this microarchitecture level, like the system-level simulation?

At the RTL level, who will code the design block? Is it the block-level designer? What information goes into the coding as in the block I/O description? Decisions like whether to instantiate or use inference, whether to re-use code, or create from scratch, need to be made at this level.

And the validation at this level would be the RTL simulation.

Design Challenges

Design

- Size and complexity

Multimillion gate designs, dividing up the design and delegating the work, and interfacing between the divided blocks.

- Power

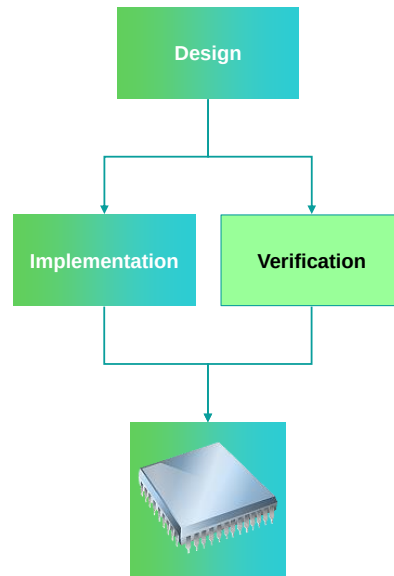
Power requirements have to be considered in the design process.

- Performance

High clock frequencies mean the designers must consider the technology information during the design process.

- IP

Choosing the right IP from the right source for the right application and technology.



So, what are the Design Challenges one has to face?

- Size and complexity

Multimillion gate designs, dividing up the design and delegating the work, and interfacing between the divided blocks.

- Power

Power requirements have to be considered in the design process.

- Performance

High clock frequencies mean the designers must consider the technology information during the design process.

- IP

Choosing the right IP from the right source for the right application and technology.

Flow Example

Let's take a simple example through the flow phases we have seen so far.

We will cover each step and highlight the following:

- Definition and step in the overall flow
- Inputs and outputs
- Formats
- Example per step



Now let us take an example and go through the different phases that we discussed just now and then go on to the next phase in the design cycle, which is Implementation.

In the next few slides, we shall cover an example that shall explain very clearly the different phases we covered till now, namely, Design Specification, defining microarchitecture, coding RTL, and then running simulation.

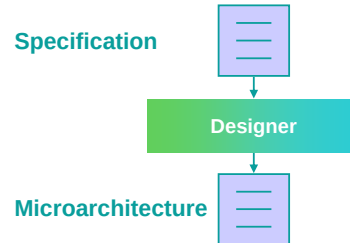
So, we shall essentially be covering and highlighting the following concepts:

- Definition and step in the overall flow
- Inputs and outputs
- Formats
- Example per step

Specification and Microarchitecture: Input and Output, Format

Specification

- Input: Requirements from Marketing, CEO (Chief Executive Officer), CTO (Chief Technology Officer), etc.
- Output: Document or model in text/graphics or software (C++, SystemC, SystemVerilog, etc.) format.



Microarchitecture

- Input: Specification + requirement from the designer.
- Output: Typically, a document in text/graphics could be software as well.

23 © Cadence Design Systems, Inc. All rights reserved.



This slide shows the first 2 phases, the specification creation, and the microarchitecture creation.

Specification needs requirements from the Marketing, the CEO, or the Chief Executive Officer, as well as the CTO, or the Chief Technology Officer, and others too as required.

Here the input is the requirements as mentioned by all these stakeholders, and the output will be a document or model in terms of text, graphics or software, for example, in C++, SystemC, SystemVerilog, etc.

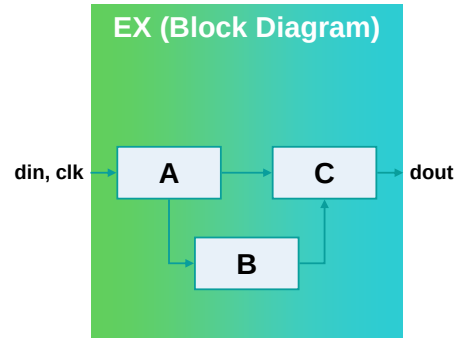
For the microarchitecture phase, the input will be the specification document created in the first phase we just mentioned, as well as the requirement from the designer. Then the output will typically be a document in text and/or graphics, or even the software itself.

Example: Specification

Let's assume we have a specification, microarchitecture, and RTL.

We are designing a chip called "EX" with:

- Three main partitions "A," "B," and "C."
- Memories in each partition.
- Perimeter I/O.
- 250-MHz clock.
- 10W total power.
- Die size not to exceed 10x10 mm² due to custom package requirements.



Let's look at an example now.

Let's assume we have a specification, microarchitecture, and RTL.

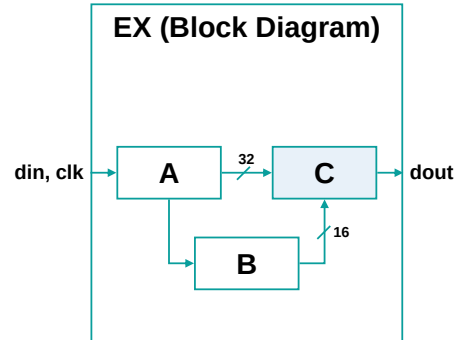
We are designing a chip called "EX" with:

- Three main partitions "A," "B," and "C."
- Memories in each partition.
- Perimeter I/O.
- 250-MHz clock.
- 10W total power.
- Die size not to exceed 10x10 mm² due to custom package requirements.

Example: Microarchitecture

For Block C

- 32-bit data bus interface to Block A.
- 16-bit control interface from Block B.
- Use 64 Mb of SRAM.
- Duplicate datapath elements in a parallel implementation.
- Limit of five clock cycles from data input processed to data output.



Now at the microarchitecture level, out of the 3 sub-blocks in the design, let's take the example of Block C.

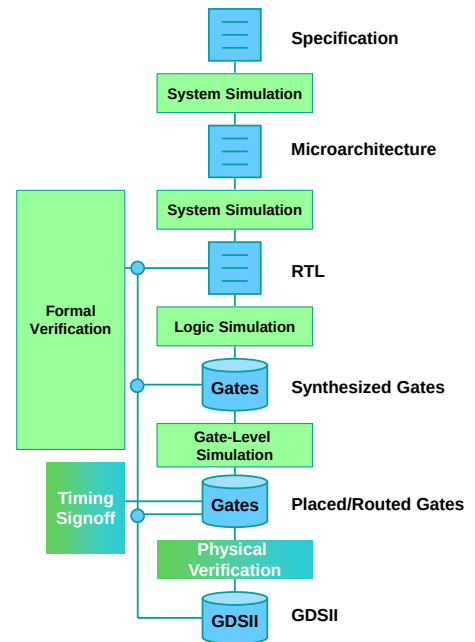
Block C has the following Micro-architecture details:

- *It has a 32-bit data bus interface to Block A.*
- *A 16-bit control interface from Block B.*
- *It uses 64 Mb of SRAM.*
- *It duplicates datapath elements in a parallel implementation.*
- *And has a Limit of five clock cycles from data input processed to data output.*

Verification

Various resources necessary to translate the representations through the verification flow.

System Simulation	Simulation at the behavioral or system level.
Logic Simulation	Simulation at the RTL level.
Gate-Level Simulation	Simulation of the discrete logic.
Physical Verification	Design rule checking, and layout versus schematic checking.
Formal Verification	Logical equivalence checking.
Timing Signoff	Static timing analysis.



26 © Cadence Design Systems, Inc. All rights reserved.

This slide shows the different kinds of testing or verification done at different phases of the design flow. We have the necessary resources in place at each of the phases accordingly.

For the simulation at the behavioral or system level, we have System Level Simulation as the process to go to the next phase in the verification flow.

At the RTL level, we have logic simulation.

We need gate-level simulation to verify the discrete logic.

For design rule checking and layout versus schematic checking, we have physical verification.

We have formal verification for logical equivalence checking

And we need to have a timing signoff for Static Timing Analysis.

Verification Challenges

Verification

- Size and complexity

Larger and more complex designs require sophisticated verification strategies.

- Testbenches

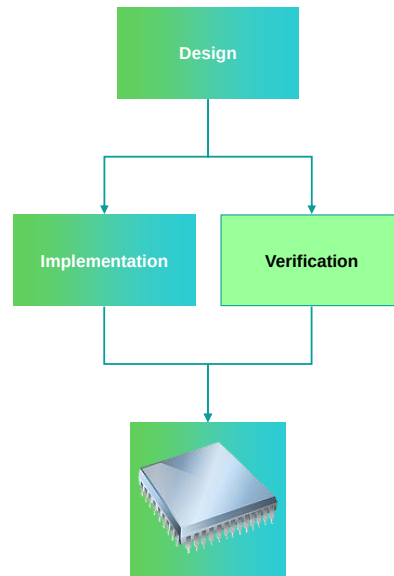
Designers need to create environments to fully test the behavior of their system.

- Corner cases

Designers need to understand the complete behavior of their system, including all of the exception cases, so they can test for them.

- Design for test

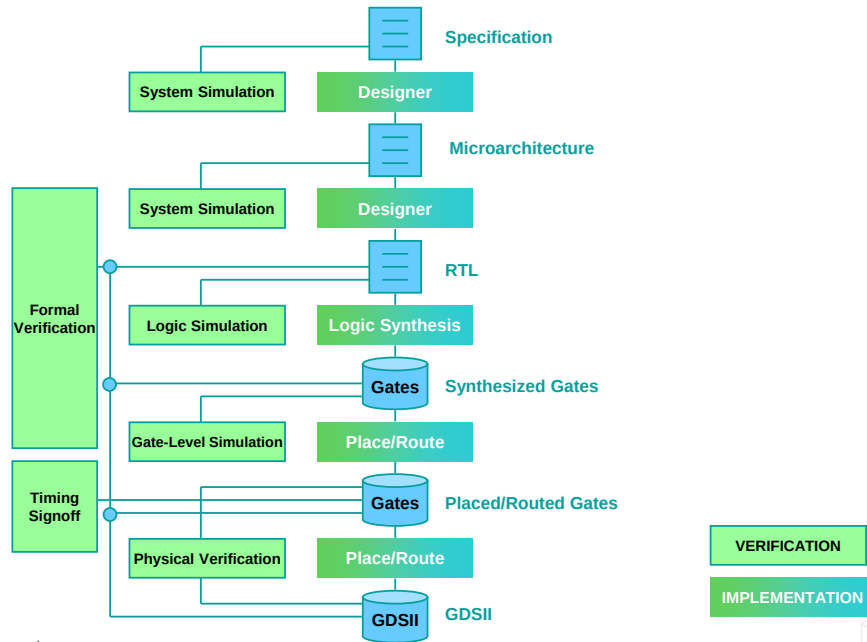
Designers must consider the strategies for testing the final product.



Just like the design challenges we listed out, we have verification challenges too!

- In regards to size and complexity, larger and more complex designs require sophisticated verification strategies.
- Designers need to create environments, which is the complete infrastructure as in testbenches and related components, to fully test the behavior of their system.
- Designers need to understand the complete behavior of their system, including all of the exceptions and corner cases, so that they can test for them.
- Designers must consider the strategies for testing the final product, which constitutes the Design for the test phase.

Implementation and Verification



28 © Cadence Design Systems, Inc. All rights reserved.

Apart from verification, the other process which runs in parallel throughout the design flow at different phases is the implementation process.

Each phase of the verification will have a corresponding implementation phase, as seen in the flow diagram on this slide.

We have logic synthesis after the Logic simulation process successfully passes.

We have place and route along with gate-level sims, which are run on the verification side.

GDSII is the final output after place and route post-physical verification.

Summary

In this module, we

- Drew a flow diagram of the entire design flow.
- Identified the distinction between Digital IC design, verification, and implementation.

Our exploration of the Digital Design Flow continues in Part 2

